1 Deep Learning for Navigation in Robotics

End-to-end supervised learning is investigated in [1] using a CNN. An end-to-end approach is comparatively simple to other deep learning approaches as it directly maps raw sensor inputs to actions. However, the paper outlines that 7202 images were used for training and that data was collected by manually operating the robot, resulting in time-consuming training. Due to this method, this time is made up by the fact that CNN technology, particularly the use of AlexNET (as outlined in the paper), has a fast inference time - a useful feature especially in potentially dangerous situations such as vehicle navigation. Reviewing the results [1, p. 4], this method allows robot navigation to almost precisely mimic human navigation when faced with both trained and untrained obstacles, as shown by the small error of linear and angular velocities. A possible downside is that the model is trained on only a chair and a human and the results are limited for untrained objects. Codevilla et al. [2] explores dataset bias in behaviour cloning and that such models are susceptible to spurious correlations (causal confusion) that may not be appropriate in different environments, implying that the model by Kim et al. would struggle to generalise beyond the trained dataset. Such a method would be useful for warehouse robots, where the robot's environment (layout and obstacles) does not change, and training data are readily available.

Deep Reinforcement Learning is reviewed in [3]. For navigation around obstacles, DRL uses sensors such as laser range finding and cameras. Unlike Monte Carlo Localization, which suffers from sensor noise, DRL has low dependence on sensor accuracy [3, p. 6], meaning the errors we see in the more classical methods do not influence this method so greatly. In addition, DRL has the benefit of a high learning rate, as well as being mapless (there is no requirement for a predefined or explicit map of the environment), allowing for more generality. A potential downside for this method is the requirement of a simulation environment for learning, as this trial-and-error method will result in the robot colliding with obstacles during training. Whilst this may take some time to set up, it could be more efficient than collecting a large dataset of images (as seen in the previous example). Indeed, as already stated, this method is associated with a high learning rate and so once a suitable environment is created, this method is efficient for training and it requires less human interaction than that seen in the previous example. The robot trained on this model is able to navigate successfully in situations with multiple robots due to the low randomness of its actions [3, p. 7], yet in social navigation environments - arguably a more critical context - where unpredictable movements require finer decision-making, the model struggles, leading the authors to conclude that "Ensuring safe crowd navigation requires further research"[3, p. 11].

DeepSLAM is an unsupervised deep learning visual SLAM requiring only stereo imagery instead of annotated ground truth poses[4], allowing for easier construction of a relevant dataset. The system trains Tracking-Net (an RCNN and RNN for estimating poses) and Mapping-Net (used for producing depth maps) unsupervised using stereo images. Whilst only monocular images are required for testing, stereo images used during training are for the purpose of recovering scale information. DeepSLAM appears to perform very well, even in challenging scenes (especially compared to LSD-SLAM and ORB-SLAM-S) as the path taken by the robot is almost identical to the path created by GPS [4, p. 8]. It's worth noting that the RobotCar dataset was used for testing, a different set to that used in training. In [5], it is highlighted that the challenge with many visual SLAMs arises from solving the loop closure problem as a result of environment problems - DeepSLAM appears to have overcome this issue. Thus, unlike that observed in the previous two methods, this method appears to generalise well to untrained environments. However, [5] also states that visual SLAMs are computationally expensive, and DeepSLAM is no different, especially considering the use of RCNN and RNN, both of which are computationally intense.

Self-Supervised learning uses curiosity-driven exploration as stated in [6]. The paper states that extrinsic rewards (which many other deep learning methods use for training) are often sparse and difficult to construct. This is because, in most cases, an agent is required to collide with something (which would not be safe in the real world) or reach an end goal - perhaps by random navigation - in order to update its policy. As an alternative, they suggest that a curiosity driven approach can overcome this issue, an approach influenced by human psychology [6, p. 1]. The intrinsic reward system should allow the agent to explore effectively without external supervision, by encouraging the agent to visit novel or unfamiliar states [7], or by motivating it to perform actions that reduce uncertainty in its own predictive models [6]. This approach was explored in two different games, Super Mario Bros (a 2-D space) and VisDoom (a 3-D space). For the application of this technology to real world scenario, VisDoom is a more appropriate scenario due to its 3-D nature. The results from testing on VisDoom show that the curious agent is able to learn faster than the non-curious agent [6, p. 5], which could suggest that such a curious model may be able to navigate untrained environments more efficiently. In order to under-

stand if this is the case, [6] explores generalisation by applying the model to untrained environments. From [6, p. 7], we see that the curiosity model is indeed able to generalise to untrained environments. However, a demonstration of this model [8], shows that while the agent is able to navigate well, it often does so by colliding into walls. As a result, further development is required in order to ensure this model is safe for use in real-world environments.

2 Analysis of hyper-parameters for CIFAR100

2.1 Introduction

The CIFAR100 dataset contains 100 classes with 600 images for each. 500 images per class are used for training, with 100 images per class for testing [9]. This report uses a modified version of DenseNet [10], which has previously been applied to CIFAR100. DenseNet has shown strong performance on CIFAR datsets, making it a viable choice for this report. Due to the fact we are using the Sequential API - for ease and speed of hyper-parameter tuning - building a true DenseNet is not possible as the Sequential API has no mechanism to access earlier layers for dense connectivity. In addition, we have added dropout, as one of the hyper-parameters to explore for training on CIFAR100, in order to effectively control over-fitting. The other hyper-parameters we will investigate are the weight decay (mimicking [10]) and growth rate.

As CIFAR100 is smaller than some of the datasets used in the paper (i.e. ImageNet), this report uses a smaller network: 7 convolution layers with a batch normalisation layer and ReLU activation layer after each, in addition to 3 dropout layers, one pooling layer and finally a dense layer. Furthermore, we are using a significantly lower number of trainable parameters, improving training time and reducing the risk of over-fitting. This lightweight architecture is well-suited to CIFAR100 as each image is only 32x32 RGB. The dataset is preprocessed prior to training, with all images being converted to 32-bit floating point values and normalised to the [0,1] range by dividing by 255, improving convergence. Moreover, the class labels have been one-hot encoded to create a 100-dimensional binary vectors, which is required for training with a softmax output layer.

2.2 Method

During testing, two hyper-parameters were kept fixed, while the third was varied. The model was trained 5 times, and an average was taken, in order to account for the effect of randomly initialised weights at the start of training. Each model was trained for 100 epochs, with a batch size of 32 and validation split of 0.2. While DenseNet [10] uses 300 epochs and WideResNet [11] uses 200 epochs , the choice of 100 epochs is in line with standard practice for models trained with Adam without the advanced learning rate scheduling seen in these examples. The code file and full results used for this report can be found at https://github.com/Dillon-Morrey/34212-Lab

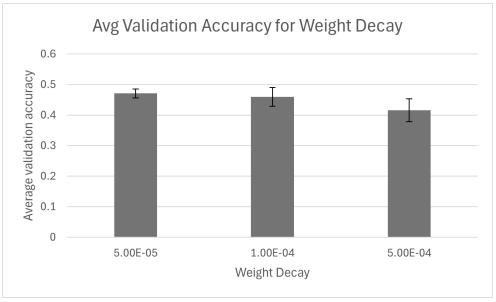
2.3 Results

2.3.1 Weight Decay Tuning

To investigate the impact of L2 regularisation, three values of weight decay were tested: 5×10^{-5}, 1×10^{-4} and 5×10^{-4}, with dropout rate and growth rate set constant with values 0.3 and 32 respectively. The results from this test are shown in Figure 1. As shown, a weight decay of 5×10^{-5} gave the highest average validation accuracy of $\approx 47.1\%$, with a low variability. As weight increased, the average validation accuracy declined, with the most significant drop in accuracy of $\approx 4.41\%$ between the weight decay values of 1×10^{-4} and 5×10^{-4}. These results suggest that excessive regularisation limits the CNN's ability to learn the required features. Figure 2 shows the training and validation accuracy over 100 epochs for each value of weight decay. These graphs are generated from a single run, rather than an average over the 5 runs since this would require taking an average value after every epoch. As a result, the randomly initialised weights do contribute more significantly to these results. As weight decay increases, we see that the training curve flattens out after less epochs, resulting in a lower final accuracy, potentially indicating under-fitting due to to over-regularisation. In contrast, with a weight decay of 5×10^{-5}, we see a consistent gap between train and test accuracy, indicating healthy generalisation. Moreover, the training curve for this value appears to still be increasing, and so may benefit for additional epochs. The results from both figures suggest that 5×10^{-5} provides the best accuracy for this model, and that while regularisation can be used to improve generalisation, larger values of this parameter can limit the model from fitting to the trained data.

2.3.2 Dropout Rate Tuning

Dropout is also used to prevent over-fitting. To assess it's impact, dropout rates of 0.2, 0.3 and 0.5 were tested, with a constant weight decay and growth rate set to 5×10^{-5} and 32 respectively. The results from the tests are shown in Figure 3 and Figure 4. To understand the effect of dropout rate, we will consider both figures simultaneously. A dropout rate value of 0.5 yields very little over-fitting as shown by the fast convergence of the test

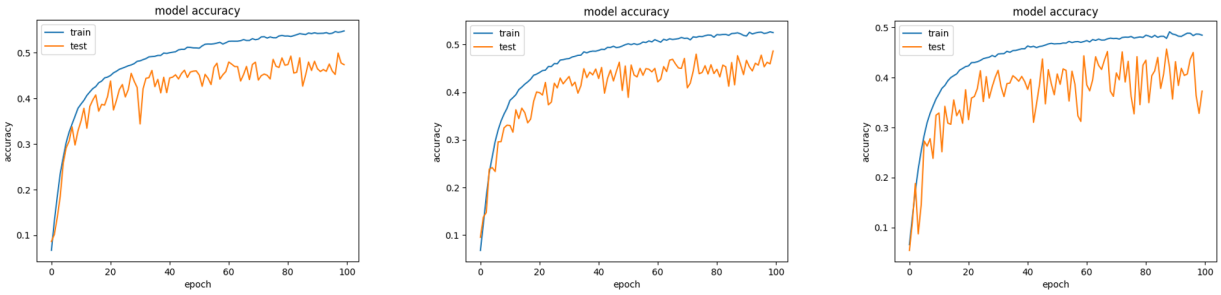


(a) Validation accuracy for different weight decay values.

Weight Decay		
Weight Decay	Avg Validation Accuracy	St Deviation for Average
5.00E-05	0.47072	0.014207639
1.00E-04	0.45974	0.030644135
5.00E-04	0.41562	0.037600359

(b) Summary of average validation accuracy and standard deviation

Figure 1: Comparison of model performance for different weight decay values.



(a) Weight decay $= 5 \times 10^{-5}$ (b) Weight decay $= 1 \times 10^{-4}$ (c) Weight decay $= 5 \times 10^{-4}$

Figure 2: Training and validation accuracy curves for different weight decay values.

line to the train line in Figure 4c, yet it resulted in the least accuracy among the three values as shown in Figure 3. Such a high value has limited the models ability to recognise patterns in the training dataset, resulting in this observation as shown again by the train line in Figure 4c. Conversely, for a dropout rate value of 0.2, we see more over-fitting from Figure 4a. However, as seen in Figure 3 this configuration still achieved relatively high validation, as the model was able to learn better representations. As indicated in Figure 3, a dropout rate of 0.3 yields the best results for this model. The combination of both figures indicates this value provided the best balance between regularisation and model capacity. Figure 4b shows a consistent gap between training and validation, implying that the model is generalising well.

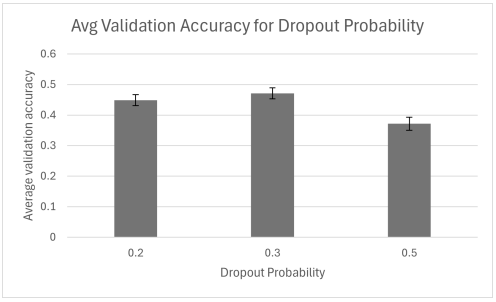
2.3.3 Growth Rate

To test this hyper-parameter, the values 16, 24 and 32 were used with dropout rate and weight decay fixed to 0.3 and 5×10^{-5} respectively. Growth rate impacts the number of new feature maps added in each layer, with higher growth rate resulting in an increase to the model's representational capacity, but raise computational cost and risks over-fitting. However, as seen in Figure 6, over-fitting does not appear to be an issue in this model due to the convergence of the two lines in each sub-figure. This is likely due to the use of weight decay and dropout within the model. An increase in growth rate consistently shows greater accuracy, as indicated by Figure 5, with the optimal value of 32. This result may be influenced by the nature of the CIFAR100 dataset, which contains a limited number of images per class, making this is a difficult dataset to learn. The high growth rate allows the model to extract rich features, which is important for generalisation.

2.4 Summary and Future Work

This experiment consisted of evaluating three hyper-parameters: weight decay, dropout rate and growth rate. For dropout rate, while both 0.2 and 0.3 attained high validation accuracy, 0.3 showed less over-fitting, making it the optimal value for this parameter. Meanwhile, the optimal value for weight decay was 5×10^{-5}, due to its ability to prevent over-fitting without significantly impacting learning. Finally, it was discovered that increasing growth rate consistently improved accuracy, with an optimal value of 32, allowing the model to capture more features. A summary of results can be found in Table 1

For future work, one could experiment with higher growth rate values, to investigate where the pattern identified in this report breaks down, if at all it does. Further to this, one could experiment with other parameters. It was mentioned that some tests in this report may benefit from more epochs, this is just one parameter that could be experimented with. Finally, one could change the CNN model to more closely resemble the DenseNet by using the Functional API, allowing the CNN to be truly dense, which is likely to provide better accuracy.

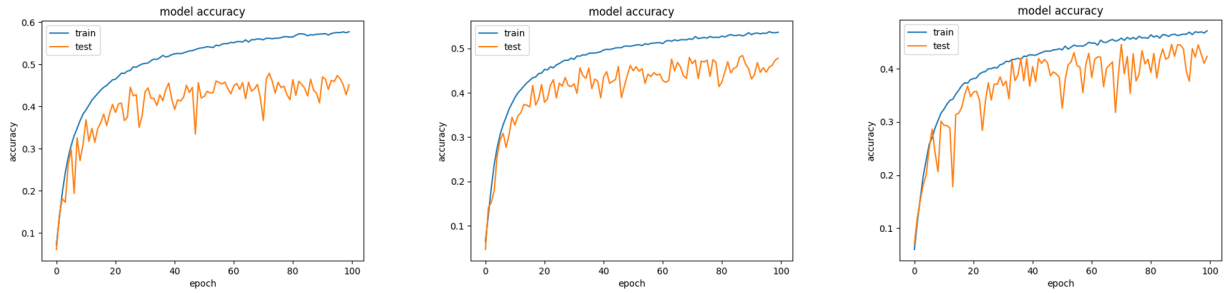


(a) Validation accuracy for different dropout rate values.

Dropout Probability	Avg Validation Accuracy	St Deviation for Average
0.2	0.44852	0.017965857
0.3	0.4707	0.01794199
0.5	0.37144	0.021654399

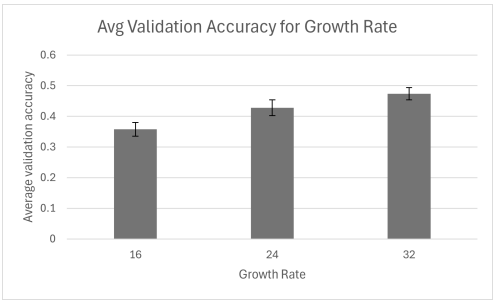
(b) Summary of average validation accuracy and standard deviation

Figure 3: Comparison of model performance for different dropout rate values.



(a) Dropout rate = 0.2 (b) Dropout rate = 0.3 (c) Dropout rate = 0.5

Figure 4: Training and validation accuracy curves for different dropout rate values.

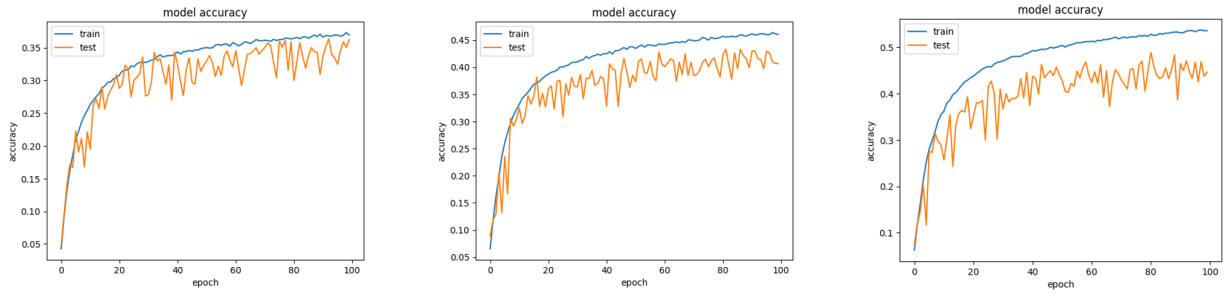


(a) Validation accuracy for different growth rate values.

Growth Rate	Avg Validation Accuracy	St Deviation for Average
16	0.35772	0.022219968
24	0.42802	0.026211295
32	0.4736	0.020317849

(b) Summary of average validation accuracy and standard deviation

Figure 5: Comparison of model performance for different growth rate values.



(a) Growth rate = 16 (b) Growth rate = 24 (c) Growth rate = 32

Figure 6: Training and validation accuracy curves for different growth rate values.

Hyper-parameter	Values Tested	Best Value	Justification Summary
Dropout Rate	0.2, 0.3, 0.5	0.3	Best balance of under-fitting and and over-fitting
Weight Decay	5×10^{-5} to $5 \times 10-4$	$5 \times 10-3$	Strong performance, no over-regularisation
Growth Rate	16, 24, 32	32	More features extracted; beneficial for CIFAR-100

Table 1: Hyper-parameter Tuning Results

References

[1] Ye-Hoon Kim, Jun-Ik Jang, and Sojung Yun. End-to-end deep learning for autonomous navigation of mobile robot. In *2018 IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–6, 2018.

[2] Felipe Codevilla, Eder Santana, Antonio M. Lopez, and Adrien Gaidon. Exploring the limitations of behavior cloning for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[3] Kai Zhu and Tao Zhang. Deep reinforcement learning based mobile robot navigation: A review. *Tsinghua Science and Technology*, 26(5):674–691, 2021.

[4] Ruihao Li, Sen Wang, and Dongbing Gu. Deepslam: A robust monocular slam system with unsupervised deep learning. *IEEE Transactions on Industrial Electronics*, 68(4):3577–3587, 2021.

[5] Iman Abaspur Kazerouni, Luke Fitzgerald, Gerard Dooly, and Daniel Toal. A survey of state-of-the-art on visual slam. *Expert Systems with Applications*, 205:117734, 2022.

[6] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2778–2787. PMLR, 06–11 Aug 2017.

[7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[8] Pulkit Agrawal. Curiosity driven exploration demo. https://www.youtube.com/watch?v=J3FHOyhUn3A&t=92s, 2020. Accessed: 2025-03-27.

[9] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical Report TR-2009, University of Toronto, Toronto, Canada, 2009.

[10] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[11] Sergey Zagoruyko and Nikos Komodakis. Diracnets: Training very deep neural networks without skip-connections, 2018.